GazeSync: Reproducible Synchronization of Screen Interaction and Multi-Source Eye Tracking

Yaxiong Lei
University of St Andrews and University of Essex
St Andrews / Colchester
United Kingdom

Shijing He
King's College London
London, United Kingdom

Xinya Gong
University of St Andrews
St Andrews, United Kingdom

Juan Ye
University of St Andrews
St Andrews, United Kingdom

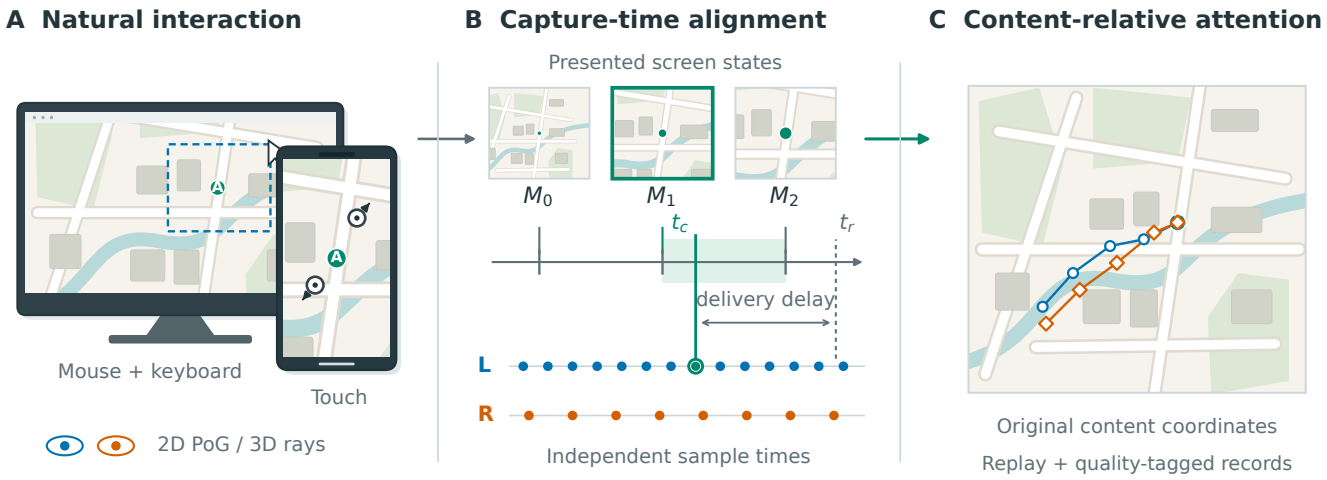


Figure 1: GazeSync links natural interaction to content-relative attention. (A) Desktop and mobile views change through mouse, keyboard and touch input. (B) Each eye retains independent timestamps; the screen state at capture, M_1, is selected despite later delivery. (C) Gaze is reconstructed on the original content, with raw records and quality decisions retained. The map views and sample paths are schematic.

Abstract

A gaze sample and a zoom event can arrive together even though they describe different screen states. Recovering attention on manipulated content therefore requires explicit temporal and spatial provenance. We extend GazeSync into a standalone Python and TypeScript framework for desktop and mobile interaction, 2D points of gaze, 3D eye-origin rays, and independent monocular or binocular streams. The framework separates capture, input, presentation and receipt time; joins each observation to a surface-specific presentation history; and preserves uncertainty, eye identity, geometry and missing data in a replayable record. SDK-independent adapters normalize model and tracker records while preserving source cadence and confidence semantics. A reproducible synthetic matrix spans 78 configurations and 107,070 observations, including 30–240 Hz displays, 30–2,000 Hz gaze and nine viewport sizes. Both implementations select the reference capture frame in every case; maximum cross-language viewport disagreement is 4.55×10^{-13} logical pixels. The results verify temporal association and coordinate reconstruction under known inputs. The public implementation includes source adapters, structured diagnostics and replay tools for instrumenting research applications with manipulated visual content.

Keywords

eye tracking, temporal synchronization, pan and zoom, binocular gaze, reproducibility, research software

1 Introduction

A person zooms into a map landmark, rotates the map, drags it, and then resets the view. A screen-coordinate gaze point can refer to different content locations during that sequence. A delayed tracker callback adds another ambiguity: the view visible when the callback is handled may differ from the view at exposure. On a phone, simultaneous finger movement and changing display pose introduce further spatial dependencies. Researchers need to distinguish these effects before interpreting a trace as attention to a particular feature.

The original GazeSync introduced mobile gaze and image-transform logging with content-relative reconstruction [1]. Within the broader mobile-gaze pipeline of sensing, estimation and interaction reviewed by Lei et al. [2], this extension addresses the association of gaze observations with rendered content. It defines how a renderer and heterogeneous gaze sources exchange timing, geometry and validity, then provides matching Python and TypeScript implementations. Researchers integrate the framework into

Table 1: Implemented contracts and verification methods.

Layer	Contract and evidence
Interaction	Wheel, drag, box and keyboard; pinch, twist and free movement. *Evidence:* gesture and browser regressions.
Signals	2D PoG, 3D rays and direction-only records; independent or combined eyes. *Evidence:* analytic and source fixtures.
Alignment	Capture-time presentation history, uncertainty and quality flags. *Evidence:* conformance and exact replay.
Integration	SDK-independent model and tracker records. *Evidence:* synthetic protocol fixtures.
Distribution	Python and TypeScript, zero runtime dependencies. *Evidence:* clean package installations.

their application and supply its input events, rendered states and gaze samples.

We contribute a capture-time synchronization contract and matching implementations that retain independent eye channels, explicit geometry and source provenance. A deterministic simulation suite, cross-language conformance checks and replay make reconstruction decisions reproducible. Source adapters and structured diagnostics support integration with gaze models and hardware trackers. This report documents implementation 0.5.3 and recording schema 2.1.

2 Background and scope

2.1 Relationship to the original GazeSync

The prior system used a Flutter/iOS application with an on-device gaze SDK and local logging server. Its callback pairing and nearest-state merge used a 50 ms tolerance. A three-participant formative study illustrated both useful content-relative reconstruction and calibration or compound-transform limitations [1]. The present implementation develops the temporal and geometric contracts behind that reconstruction.

The extension replaces the platform-specific merging assumption with explicit presentation intervals, source clocks and quality decisions. It adds desktop interaction, independent eye identity, metric ray projection, reusable adapters and a common record format across languages. Table 1 summarizes the implemented contracts and their verification.

2.2 Interaction and synchronization systems

Gaze-supported pan and zoom has been investigated as an interaction technique, including combining gaze with other input modalities [6]. DynamicRead evaluated gaze-based scrolling for handheld reading in sitting and walking conditions, finding differences in the techniques' robustness under mobility [3]. Our instrumentation records manual interaction and gaze as separate streams. A person may inspect a landmark while it moves, look at a stable screen location while dragging, or shift attention ahead of the next action. These scenarios motivate the attention policies used in the simulation.

Lab Streaming Layer provides timestamped multimodal streams and network clock synchronization [7]. Such infrastructure can supply a transport or clock estimate to a GazeSync host. GazeSync uses the supplied clock mapping to select the rendered content state, reconstruct coordinates and identify ambiguous associations. Device-specific data contracts also matter: Tobii display coordinates and metric user coordinates differ from Pupil scene-camera coordinates [8, 9]. Adapters make these coordinate and timestamp conversions explicit.

3 Design requirements

Record what changed on screen. Mouse, keyboard and touch events express intent. A renderer can coalesce many inputs into one display update or animate a single command across several frames. We record input events for explanation and presentation snapshots for reconstruction. A snapshot includes surface, layout epoch, content identity, viewport dimensions, the rendered affine matrix, timing provenance and optional registered display geometry. Layout changes and suspension are observable events.

Respect independent measurements. A binocular tracker can lose one eye, sample eyes at different times, or expose a fused output with different semantics. The record retains `left`, `right`, `combined` or `unknown` identity. Separate observations retain their timestamps and validity. Independent and combined channels remain distinct, with validity stored separately from scalar confidence.

Keep disagreement visible. Missing history, uncertain clocks, incompatible coordinates and invalid samples must remain inspectable. A seemingly smooth trace can conceal an acquisition error. Raw data and quality flags are retained alongside derived coordinates; blinks and gaps break a plotted trace. Acquisition rates and quality thresholds are configurable for each study.

Keep adaptation local and testable. The synchronization core depends on neither a hardware SDK nor a learned model. Adapters convert explicit records at the edge, while a single finite-JSON contract carries the result. This permits hardware or inference changes without changing the content-reconstruction rules. Cross-language fixtures check that Python analysis and TypeScript acquisition interpret the same record consistently.

4 Temporal reconstruction

4.1 Four event times and a clock epoch

We distinguish input time t_i, presentation time t_p, source capture time t_s and session receipt time t_r. A frozen mapping for clock epoch e converts source time to session capture time:

$$t_c = b_e + r_e(t_s - a_e), \tag{1}$$

where a_e and b_e are corresponding anchors and r_e is the clock-rate ratio. Canonical times are monotonic milliseconds. Browser timing follows the monotonic clock model of High Resolution Time [13]; an epoch wall clock cannot be substituted without a measured mapping.

The host supplies clock uncertainty and optional drift uncertainty. The latter grows with distance from the anchor; conversion

also includes a conservative floating-point term. A revised fit or source restart requires a new epoch. These engineering bounds describe the host's clock calibration. Receipt-only sources remain recordable but are excluded from capture-time attention analysis.

4.2 Presentation intervals and ordering

For the surface-specific history $\mathcal{H}_s$, alignment selects

$$j^* = \underset{j \in \mathcal{H}_s:\, t_j^p \le t_c}{\arg\max} \; (t_j^p, \text{sequence}_j). \tag{2}$$

The last submitted snapshot wins a timestamp tie; superseded equal-time snapshots have no presentation interval and do not create temporal ambiguity. Each effective snapshot persists until its successor. Geometry is not interpolated through an unobserved animation, reset or content replacement. The host must record every relevant intermediate rendered state, or explicitly assert change-only presentation logging.

A call to `flush(h)` is a watermark: it declares that all presentations through h have already been submitted. Insertion at or before that watermark is rejected. Samples normally wait for the greater of the configured reorder window and maximum accepted clock uncertainty. Ready samples are sorted by capture time within each flush. Later arrivals remain identifiable even when retained history permits valid offline reconstruction. Impossible future times and excessive uncertainty are quarantined promptly, preventing indefinite pending-buffer growth.

The interval $[t_c - u_c, t_c + u_c]$ is checked against visual changes. If it crosses a relevant change, the output is marked ambiguous. Boundaries that repeat identical geometry do not create false ambiguity. Histories are indexed per surface; retention preserves the newest state of each active surface, so frequent updates on one display do not evict another display's only state. A required older state may still expire, producing an explicit history flag.

4.3 Why timing errors become spatial errors

For a viewport observation p_v and rendered matrix $M(t)$, content gaze is $M(t)^{-1}\widetilde{p}_v$. Away from discontinuities, a small timing displacement Δt yields a first-order spatial error proportional to

$$\left\| \frac{dM(t)^{-1}}{dt} \widetilde{p}_v \right\| |\Delta t|. \tag{3}$$

Fast zoom, rotation or translation can therefore amplify a modest delivery delay. At a reset or layout discontinuity this derivative approximation does not apply; the correct presentation interval must be selected explicitly. Figure 2 illustrates this mechanism with known synthetic timing. The receipt-time baseline isolates the effect of selecting the view at delivery rather than at capture.

5 Spatial reconstruction and eye identity

5.1 Named frames and distinct origins

We use column vectors and write T_{BA} for a change of coordinates from frame A to frame B. Each 3D observation and plane carries a `frameId` referring to a calibrated frame with a documented origin, axis directions, handedness, units and time reference. Table 2 lists these conventions.

The camera's optical centre differs from its image principal point, which is an intrinsic calibration parameter. A gaze ray's origin is the measured eye centre expressed in the chosen frame, generally not that frame's origin. Zero is valid for an eye-local origin only when its relation to the display is known. Vendor conventions also differ: Tobii UCS is centred on the tracker front surface, with user-right/up/toward-user axes and millimetre units [8]. Pupil's scene-camera frame and individual eye-camera frames are distinct [12]. The adapters retain these identities and convert metric units explicitly.

5.2 Rigid transformations: points versus directions

The implementation accepts a row-major 3×3 proper rotation R_{BA} and translation t_{BA} in metres:

$$T_{BA} = \begin{bmatrix} R_{BA} & t_{BA} \\ 0^T & 1 \end{bmatrix}, \qquad p_B = R_{BA} p_A + t_{BA}. \tag{4}$$

Here t_{BA} is the source origin expressed in the target frame. The rotation must satisfy $R^T R = I$ and $\det R = +1$; reflections, scaling and shear are rejected. The helpers distinguish ray origins and directions:

$$O_B = R_{BA} O_A + t_{BA}, \qquad D_B = R_{BA} D_A. \tag{5}$$

Translation must not be applied to a direction, whose homogeneous coordinate is zero. Inversion gives $R_{AB} = R_{BA}^T$ and $t_{AB} = -R_{BA}^T t_{BA}$.

OpenCV extrinsics map world coordinates into camera coordinates, $p_C = R_{CW} p_W + t_{CW}$ [11]. Consequently the camera's world position is

$$C_W = -R_{CW}^T t_{CW}, \tag{6}$$

not t_{CW}. Eye origins must be transformed independently. A display-mounted camera may observe eyes at positive camera z while their gaze directions point back toward the display with negative z; this is geometrically valid. The `transformRay` and `transformPlane` helpers check named source frames, return new values, and require the eye origin explicitly. Python provides the same operations with snake-case names.

An eye-local ray with origin zero has $O_C^e = t_{CE_e}$ and $D_C^e = R_{CE_e} D_{E_e}$, independently for $e \in \{L, R\}$. Its measured registration supplies the nonzero camera-frame eye origin.

A plane's top-left point transforms with R and t, while its right/-down axes transform with R only. A transformed plane remains marked static only if both the original plane and transformation explicitly declare time independence. Applications retain the original ray, calibration identifier, transform and pose time in sample metadata to preserve conversion provenance.

5.3 Screen plane and forward intersection

A registered active rectangle has top-left P, orthonormal right/down axes u, v and physical width/height w_m, h_m. Its local-to-reference frame is

$$T_{WS} = \begin{bmatrix} u & v & u \times v & P \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{7}$$

A marker-board pose may locate the marker centre rather than the active panel corner. A separately measured registration is then required: $T_{CS} = T_{CM} T_{MS}$. The host fits a rigid pose to the known

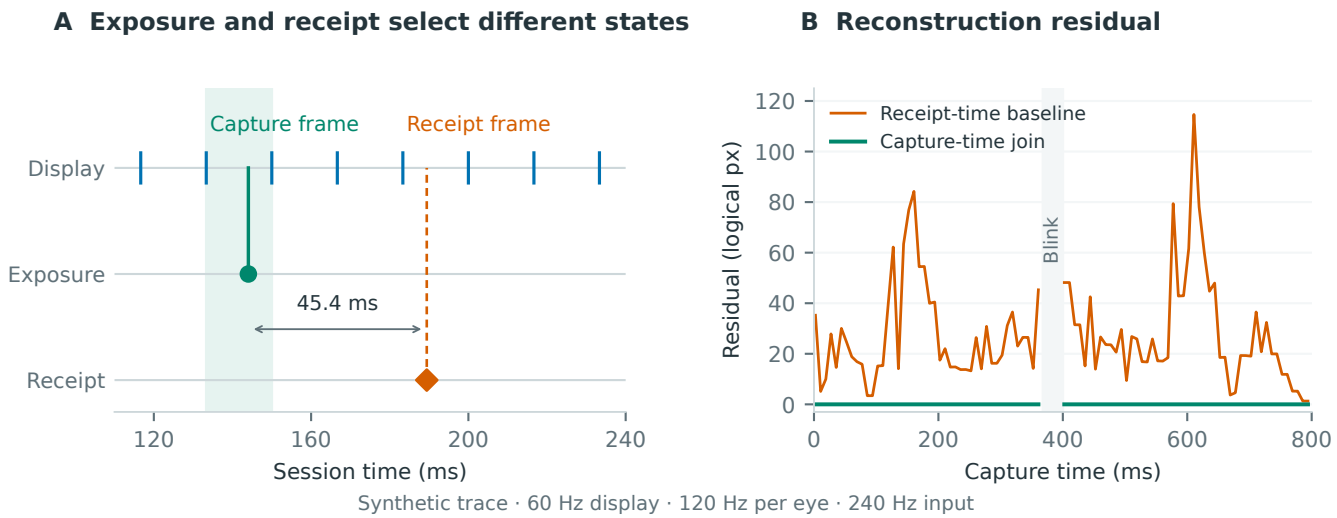


Figure 2: Observed timing in the representative synthetic trace. (A) Capture and receipt fall in different presentation intervals. (B) Receipt-time association introduces reconstruction residuals during manipulation; capture-time residuals remain at numerical scale. The blank interval denotes a simulated blink.

Table 2: Coordinate origins and conventions. Eye labels are anatomical. The optical-camera axes shown here are an explicit example; each vendor frame requires its documented conversion.

Frame	Origin	Convention and units
Camera C	Optical centre of the calibrated camera	Example: $+x$ image-right, $+y$ down, $+z$ forward; metres.
Eye E_L, E_R	Each measured eye centre	Eye-local zero requires a measured transform into C or W; metres.
Reference W	A fixed, documented host reference	Here: active-display centre at the reference pose; metres.
Screen S	Active rectangle's top-left, excluding bezel	Right u, down v, normal $u \times v$; metres.
Image	Pixel origin specified by image calibration	The principal point (c_x, c_y) is distinct from the optical centre; image pixels.
Display D	Full active display's top-left	Axes follow the current display orientation; logical pixels.
Viewport V	Stimulus region's top-left, at (l, t) in D	Parallel display/viewport axes for the offset mapping; logical pixels.
Content I	Intrinsic origin of the untransformed content	The recorded matrix M maps I to V; content units.

rectangle and supplies its physical dimensions; noisy corner differences do not automatically form an orthonormal screen basis.

For a user-right/up/toward-user reference, $u = (1, 0, 0)$ and $v = (0, -1, 0)$, so the screen-local positive normal points behind the front face. Its sign does not impose a sign restriction on a valid gaze direction.

Normalize the nonzero direction to $d = D/\|D\|$. For $n = u \times v$, the crossing is

$$\lambda = \frac{(P - O) \cdot n}{d \cdot n}, \qquad Q = O + \lambda d. \tag{8}$$

A valid forward crossing requires $\lambda > 0$. Reversing n changes numerator and denominator together, preserving λ; a negative denominator alone is not a reason to reject the ray. Zero directions, $|d \cdot n| < 10^{-9}$, non-positive distance and non-finite results are rejected. Robust direction normalization avoids overflow for very large or small vector magnitudes. The reported distance is measured from the eye, rather than camera depth.

The normalized crossing within the plane rectangle is

$$\alpha = \frac{(Q - P) \cdot u}{w_m}, \qquad \beta = \frac{(Q - P) \cdot v}{h_m}. \tag{9}$$

The operation is a two-sided geometric crossing. For an opaque panel with a known front-facing right/down basis, $(O - P) \cdot (u \times v) < 0$ places the eye in front. The host supplies front-facing and occlusion visibility and records inactive surfaces explicitly.

The infinite-plane crossing and the active rectangle are separate tests: $\alpha, \beta \in [0, 1]$ identifies the finite rectangle. Off-display intersections are retained and flagged, including when an oversized application window would contain them. A successful aligned 3D hit records its metric frame ID as well as its position and eye-to-screen distance.

5.4 Display, viewport and content coordinates

The physical rectangle represented by a plane and the logical viewport define distinct coordinate domains. If the plane describes the

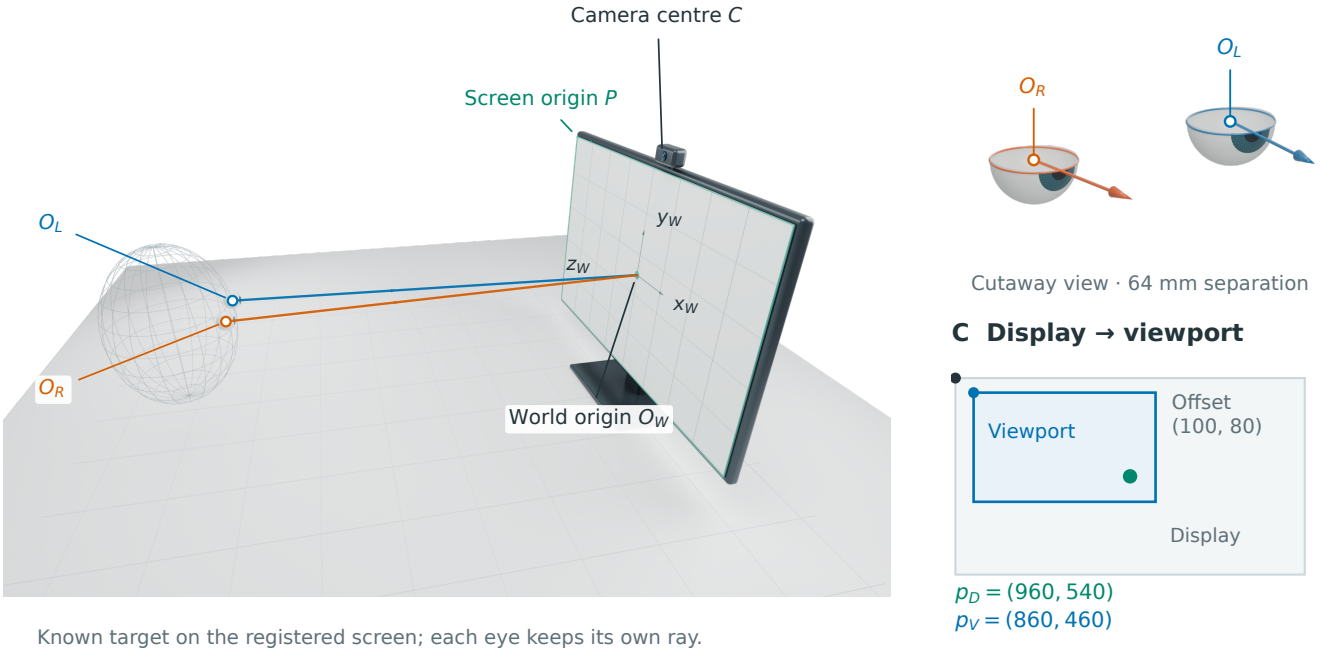


Figure 3: Independent eyes retain separate origins. (A) A procedurally rendered metric scene separates the camera centre C, active-panel origin P, world origin O_W and two eye origins O_L, O_R. Both rays independently meet the known centre target in this fixture. (B) A magnified cutaway makes the distinct eye centres visible; their separation is 64 mm. (C) The same screen hit maps from $(960, 540)$ in the full display to $(860, 460)$ in the offset viewport. Eye anatomy and viewing perspective are illustrative; labelled origins and ray endpoints follow the shared numeric fixture.

full active display, with logical dimensions W_D, H_D, then

$$p_D = (W_D\alpha, H_D\beta), \qquad p_V = p_D - (l, t), \tag{10}$$

where (l, t) is the viewport's top-left in display coordinates. This conversion is selected by `planeSpace: display` and requires explicit display layout. It matches the normalized-display PoG path.

If the plane instead describes the physical *viewport subrectangle*, then `planeSpace: viewport` gives $p_V = (W_V\alpha, H_V\beta)$ without subtracting the offset a second time. For an axis-aligned crop, the equivalent viewport plane can be constructed as

$$P_V = P_D + u\, w_m \frac{l}{W_D} + v\, h_m \frac{t}{H_D}, \tag{11}$$

$$w_m^V = w_m \frac{W_V}{W_D}, \qquad h_m^V = h_m \frac{H_V}{H_D}. \tag{12}$$

The two paths are equivalent when configured consistently. An ambiguous plane extent is rejected when the recorded viewport differs from the full display. These offset formulas assume parallel viewport/display axes. Outer application transforms must be resolved by the host; rotating the rendered content is handled separately by its affine matrix.

The content-to-viewport transform is

$$M = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}, \qquad \widetilde{p}_I = M^{-1}\widetilde{p}_V. \tag{13}$$

It includes intrinsic scaling, letterboxing, pivot translation and content rotation. Each accepted frame caches the inverse; scale-normalized conditioning checks reject singular transforms while permitting small, well-conditioned scales. Device pixel ratio is metadata, not an extra factor in Equation 10. Device-pixel measurements are normalized using their source resolution before conversion into logical units.

Canonical UI coordinates describe continuous rectangle edges from $(0, 0)$ to (W, H), not integer array indices. Pixel-centre conventions must be converted once when required by a source. Bounds classification allows arithmetic residuals of 64 machine epsilons scaled by the coordinate extent, without changing the stored coordinates.

5.5 A numerical origin and viewport check

The shared analytic fixture uses a 0.53×0.33 m panel centred at O_W, with $P_W = (-0.265, 0.165, 0)$ and camera centre $C_W = (0.04, 0.19, 0.015)$ m. It deliberately places the camera away from both panel and eye origins. With $R_{CW} = \mathrm{diag}(-1, -1, 1)$, $t_{CW} = (0.04, 0.19, -0.015)$ m. The left eye at $O_W^L = (-0.032, 0, 0.6)$ becomes $O_C^L = (0.072, 0.19, 0.585)$. A centre-directed $D_C^L = (-0.032, 0, -0.6)$ transforms to $D_W^L = (0.032, 0, -0.6)$ and intersects at $Q_W = (0, 0, 0)$. The right eye is checked independently at $O_W^R = (0.032, 0, 0.6)$.

For a 1920×1080 logical display and 1000×600 viewport at $(100, 80)$, both eyes yield $p_V = (860, 460)$. Scaling the full plane

directly to viewport dimensions would instead yield $(500, 300)$, illustrating why the registered rectangle must be explicit. With $M = \mathrm{diag}(0.5, 0.5, 1)$, the content point is $(1720, 920)$. This independent fixture checks origin translation, axis signs, units and viewport offsets.

5.6 Intrinsics and time-dependent geometry

For undistorted pinhole image coordinates $\tilde{q} = (u_{img}, v_{img}, 1)^T$,

$$Z_C \tilde{q} = K p_C, \qquad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{14}$$

The principal point (c_x, c_y) is in image pixels and differs from the optical centre $(0, 0, 0)_C$. Camera intrinsics do not replace a measured gaze origin or screen registration. A pupil image point back-projects to a camera viewing ray, not automatically the visual axis of the eye. A calibrated eye model supplies that relationship. Image resizing, cropping and distortion handling must follow the calibration convention; for an affine image-coordinate change H, the corresponding intrinsic mapping is $K' = HK$. A mirrored preview must be resolved explicitly, because a single-axis reflection is not a proper 3D rotation. Content zoom modifies M, not camera intrinsics.

For a moving phone, both ray conversion and screen pose must refer to the relevant capture time. Empirical work on mobile interactions identifies head distance, head pose and device orientation as factors associated with 2D gaze-estimation error [4]. The session uses the selected presentation's recorded pose and checks its age. The pose-age threshold limits temporal staleness; spatial error also depends on the intervening motion. Orientation changes update layout epochs and physical axes/dimensions. In the simulation, a panel's physical width and height swap with display orientation. The desktop fixture crops its viewport while retaining the active panel, top-left origin and metric pixel pitch. Physical dimensions are configurable.

5.7 Content rotation and free movement

The content layer may be an image, a fixed-layout text page or a group with a stable intrinsic coordinate system. Its top-left origin remains fixed in content coordinates while the layer moves within a clipped viewport. In the viewport's right/down axes, positive θ is clockwise and the similarity transform is

$$M = \begin{bmatrix} s\cos\theta & -s\sin\theta & t_x \\ s\sin\theta & s\cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad p_v = sR(\theta)p_i + t. \tag{15}$$

Here $s > 0$, and t is the viewport position of the intrinsic origin. The renderer uses a top-left transform origin; an additional CSS-centred origin would change this mapping. Content rotation leaves the calibrated metric screen plane unchanged. Both PoG and screen-ray intersections use the inverse of the matrix selected at capture time.

For an initial viewport anchor a, let $q = s^{-1}R(-\theta)(a - t)$. Moving the anchor to b, scaling by f and rotating by δ gives

$$s' = \mathrm{clamp}(sf, s_{\min}, s_{\max}), \quad \theta' = \theta + \delta, \quad t' = b - s'R(\theta')q. \tag{16}$$

Orientations are stored modulo 2π. Thus q stays under the new anchor before boundary constraints. A two-contact gesture derives a, b from the initial/current midpoints, f from their distance ratio and δ from the unwrapped angular change. Scale, rotation and movement are applied together before bounding once. Near-coincident contacts suspend scale and angle changes and establish a new baseline on separation; lifting one contact rebases to one-finger movement. Desktop users can drag in a rotation mode or use bracket keys for 15° steps. The API accepts radians and movement in viewport logical pixels.

Two boundary policies serve different tasks. Free movement permits repositioning at fit scale. Bounded movement clamps the axis-aligned envelope of all four rotated corners and centres an axis when that envelope is smaller than the viewport; it does not guarantee coverage of every viewport corner. Clipping can expose blank space, so outside-content flags remain part of gaze interpretation. Reset restores zero rotation, centred content and the original unrotated fit. Independently animated children, text reflow and occlusion need additional host state.

5.8 Observation identity and optional pairing

Observation identity includes source, clock ID and epoch, eye, representation and sample ID. The same physical capture can thus carry left/right and PoG/ray records without collision. Optional capture and calibration identifiers retain further provenance. Combined estimates retain their source identity. Duplicate protection covers every pending observation as well as a bounded history of recent identifiers. An identifier older than both windows can be accepted again; callers must therefore issue stable, unique sample identifiers.

For analyses that need matched eyes, an offline helper performs chronological, one-to-one pairing under a caller-specified maximum skew, defaulting to 5 ms. It considers only usable observations with matching source, representation, clock identity/epoch, surface/epoch, content identity and calibration. Unmatched observations are returned explicitly. Pairing is chronological and greedy, with each observation used at most once. Streams with different clock identities or epochs remain separate; downstream analyses can apply their own fusion policy.

6 Adapters, diagnostics and developer contract

6.1 SDK-independent normalization

Both language implementations expose generic model, Tobii Pro dictionary, EyeLink bridge and Pupil Core adapters. A generic model adapter supports nested field paths, explicit timestamp units, display-pixel normalization and metric origin conversion. Model inference and tensor transfer remain outside the core. A new adapter can be a pure function producing the shared JSON schema and structured warnings.

Tobii normalization preserves per-eye validity and normalized display points; optional rays are defined from the vendor origin to the reported display point, with millimetres converted to metres [8]. Confidence remains null when only validity is provided. The EyeLink bridge consumes serialized per-eye pixel records and millisecond device timestamps; a Python helper extracts this bridge

record from a newly received PyLink sample [10]. Hosts install the vendor SDK separately.

Pupil normalization preserves individual base-sample timestamps and deduplicates reused eye observations within a bounded history. Scene-camera coordinates require an explicit calibrated homography or registered surface record to become screen PoG. Homographies are normalized internally by coefficient magnitude and checked for rank loss before projection; the supplied coefficients remain in the observation metadata. Right/left IDs, bottom-origin coordinates and millimetre eye centres follow the vendor contract [9]. Mapping timestamp skew is retained and checked. Synthetic protocol fixtures verify these conversions.

6.2 Diagnostics as integration feedback

A source descriptor declares adapter, eye set, tracking mode, expected cadence and timestamp provenance. Expected cadence does not resample observations. Diagnostics estimate rates per source, eye, representation and clock epoch using bounded windows. They expose gaps, out-of-order delivery, rate deviations, missing history, uncertain clocks and unavailable geometry. Observers receive the first occurrence for each diagnostic context while the report retains counts and most recent occurrence times. An observer exception is recorded without interrupting acquisition.

Prediction reliability is distinct from clock and presentation uncertainty. Factor-Informed Uncertainty Distillation (FIUD) learns an image-quality-related error-ranking signal for selective gaze prediction [5]. Hosts preserve such source-specific scores and their semantics in metadata. Scalar confidence uses $[0, 1]$, clock bounds use milliseconds, and `confidence=null` records the absence of a suitable scalar confidence.

A per-observation `usable` flag and reason list remain the primary quality contract. Informational conditions, such as unavailable scalar confidence, can coexist with usable geometry. Receipt-only timestamps, blinks, missing history or ambiguous presentation make content attention unusable. Raw measurements remain available for a different downstream analysis that states its own policy.

6.3 Human interaction and host responsibilities

The controller anchors wheel zoom at the pointer, distinguishes drag from selection-box zoom, supports keyboard reset and cancels an active gesture when another view command takes over. Intervening view commands also break touch double-tap sequences. Touch interaction tracks contacts, pinch scale and midpoint translation; releasing one contact transitions to pan without a jump. Double tap, cancellation, resize, orientation change and background/resume are explicit cases. Browser handling follows pointer-event ownership and cancellation semantics [14]; it avoids capturing keys from embedded editable controls.

A convenience browser entry point, `createGazeSurface`, attaches one content layer to a viewport and handles input, CSS transforms, resize, lifecycle records and teardown. Exclusive ownership of the viewport, content layer and session prevents competing render loops. A runnable example groups an illustration with ordinary HTML text. Its controller exposes `rotate`, `move` and `reset`; the same operations are available in Python. The helper records the applied matrix in each render callback and preserves capture context for later tracker delivery. The host supplies capture timestamps, source clocks and calibrated 3D geometry.

The lower-level browser adapter records application input, while the host records what was rendered. Native Python applications can feed the same controller logic from GUI callbacks. Both cores are synchronous; applications serialize rendering and SDK callbacks through a single session owner. Hosts can record kinetic scrolling or animation through the same presentation-snapshot interface.

7 Verification and synthetic evaluation

7.1 Audit strategy and regression coverage

Regression coverage spans geometry, event ordering, browser input, recording, replay and package boundaries. Tests check surface-specific frame selection, gesture interruption, conditioned matrix inversion, transactional record validation, future-time quarantine and complete replay sequences. Further cases check content-aware eye pairing, homography scale invariance, DOM ownership and strict publication flags. Shared fixtures exercise eye identity, source adapters, receipt-time exclusion, nullable confidence and clock boundaries in both languages.

Independent geometry fixtures cover camera/eye origins, full-display versus cropped-viewport mapping, per-eye Tobii UCS offsets and non-symmetric rotations. Both languages additionally test 100 arbitrary rigid frame changes, inverse transformations, reflected/scaled matrices, direction magnitudes spanning 10^{-200}–10^{200}, dynamic-pose provenance and physical panel dimensions across orientation changes. Five native geometry fixture outputs are compared directly between implementations. These fixtures are separate from the 78-case acquisition-rate matrix.

Strict TypeScript compilation generates public declarations from the executable source. Consumer tests check exports and validity-dependent measurement types. Python tests exercise the independent implementation and validate normalized fixtures against JSON Schema. Release checks inspect imports, package contents, license consistency and shared fixtures.

7.2 Two complementary simulation families

Interaction scenarios. Desktop and mobile scenarios each last nine seconds at 60 Hz display and 120 Hz gaze, repeated with seeds 1, 41 and 90210. Desktop actions include wheel zoom, free movement, rotation, marquee selection, cancellation, reset and resize. Mobile actions include pinch in/out, two-finger twist, moving midpoint, pinch-to-pan transition, double tap, orientation and suspension. Attention policies include content-landmark tracking, screen-fixed attention during pan, scanning and reading-like fixation steps. Timing and attention trajectories are scripted test inputs. The moving-phone fixture holds pose between recorded frames.

Rate and context matrix. A second family separates frequency sensitivity from context coverage. The controlled matrix contains 42 cells: six display rates (30, 60, 90, 120, 144 and 240 Hz) crossed with seven gaze rates per eye (30, 60, 120, 250, 600, 1,000 and 2,000 Hz). It fixes a 1280×720 viewport, 240 Hz input and independent binocular acquisition. A further 36 cases cross nine viewport configurations with left monocular, right monocular, independent binocular and

combined modes. These context cases vary input rate, device pixel ratio, orientation and clock configuration together to exercise integration conditions.

The nine initial viewports range from 320×568 to 3840×2160, including portrait, landscape, tablet and ultrawide layouts. Context input rates range from 30 to 1,000 Hz. Independent context eyes use 600 and 120 Hz and separate clock mappings. Each case lasts 800 ms with seed 41. Inputs are coalesced into rendered frames. Ordinary delivery delay is 28–63 ms, with an additional 120 ms on selected captures. A blink interval occupies 46–50% of the trace. Uniform per-axis perturbation of ±0.35 logical pixels supplies source noise.

PoG and ray observations originate from the same perturbed viewport target. Reconstruction error is measured against that perturbed target using the known renderer coefficients and expressed in logical viewport pixels. This separates source noise from the residual introduced by reconstruction and tests consistency between the two representations.

7.3 Cross-language conformance and results

The test driver serializes identical events for both implementations and compares all aligned records and diagnostic reports. Keys, labels, frame identities and flags must match exactly. Numeric comparison uses absolute tolerance 10^{-8} plus relative tolerance 10^{-12}; the largest observed differences are much smaller (Table 3). The suite additionally compares ten source adapter fixtures and checks exact replay of derived alignment within the same implementation. Python-native experiment generation is exercised separately.

Across the 78 cases, all 107,070 expected observations are received, pending buffers drain, and every capture-frame selection matches the independent synthetic reference. Maximum reconstruction residual is 7.85×10^{-13} logical pixels. Maximum Python–TypeScript viewport disagreement is 4.55×10^{-13} pixels; mapped capture times agree at the reported precision. Of the received observations, 102,768 (95.98%) produce usable content coordinates. Reconstruction residuals are computed on these mapped observations against their known synthetic targets.

Additional content-motion cases. Eight further cases combine rotation and free movement in 390×844 and 1920×1080 viewports with the four eye modes. Mobile/desktop configurations use 90/144 Hz presentation and 125/1,000 Hz input, with 600 Hz primary and 120 Hz right-eye sampling where applicable. The deterministic rotation reaches ±0.8 rad; the content centre moves by up to 12% of viewport width and 10% of height while scale varies. These cases retain the 800 ms duration, noise and delivery model above; mobile cases also change display orientation. All 6,516 expected observations are received, every capture-frame choice matches, and replay is exact. Maximum mapped reconstruction residual is 8.49×10^{-13} logical pixels. These eight cases supplement the 78-case baseline.

The receipt-time baseline's P95 residual ranges from 54.4 to 79.6 logical pixels across the controlled cells (Figure 4A). This range belongs to the specified manipulation trace, viewport and delays. Raising gaze rate alone leaves the delivery-time association error in place. Frequency comparisons use the controlled cells; the context cases vary dimensions and motion together.

Table 3: Numerical consistency across 78 deterministic cases. Spatial residuals use logical viewport pixels or content units, as indicated.

Check	Result	Unit
Received observations	107,070	obs.
Usable observations	102,768	obs.
Usable coverage	95.98	%
Frame mismatches	0	obs.
Exact replay	78/78	cases
Pending after drain	0	obs.
Maximum numerical differences		
Reconstruction residual	7.85×10^{-13}	px
Python–TS viewport	4.55×10^{-13}	px
Python–TS content	4.55×10^{-13}	units
Python–TS capture time	0	ms

8 Reproducibility and distribution

The repository contains strict TypeScript source, Python source, JSON schemas, synthetic fixtures, regression tests, a local interaction lab and figure scripts. The two libraries have no runtime dependencies. The visual assets use a separate procedural figure pipeline; an editable Blender scene and projected annotation anchors reproduce the 3D geometry figure. The TypeScript distribution contains executable ES modules, generated declarations and source; the Python distribution contains a wheel and source archive. Clean installation tests run outside the checkout, including a wheel rebuilt from the source distribution.

Schema 2.1 records the implementation version, plane extent, metric hit frame, source descriptors, clock mappings, configuration, raw event sequence, retention losses and derived output. Exact replay rejects incomplete or incompatible logs, including earlier implementation versions; recordings are never silently reinterpreted under changed projection semantics. Default in-memory limits are explicit: 7,200 frames, 100,000 raw/derived records and 4,096 pending observations. Long sessions require persistence before rollover. Python raw-log storage uses a deque to avoid shifting the retained history on each insertion. Its public snapshots and JSON exports remain ordered lists. Other bounded arrays retain front-removal costs; the host manages persistence and source backpressure.

The full matrix can be reproduced with the following source-checkout commands:

```
npm run test:conformance
node tools/paper-data.mjs
.venv/bin/python tools/paper_figures.py
```

Figure generation additionally requires the pinned plotting dependencies listed in the paper build guide. The reported local run used Node.js 22.16.0 and Python 3.12.14. Continuous integration is configured across Node 22/24/26 and Python 3.10/3.12/3.14 on Linux, with Node 24 and Python 3.12 suites also passing on Windows and macOS. The aggregate JSON data preserve every configuration and result.

GazeSync is distributed through https://github.com/GazeLei/GazeSync under the GPL-3.0-only license. The repository provides both implementations, reproducible simulations, validation data

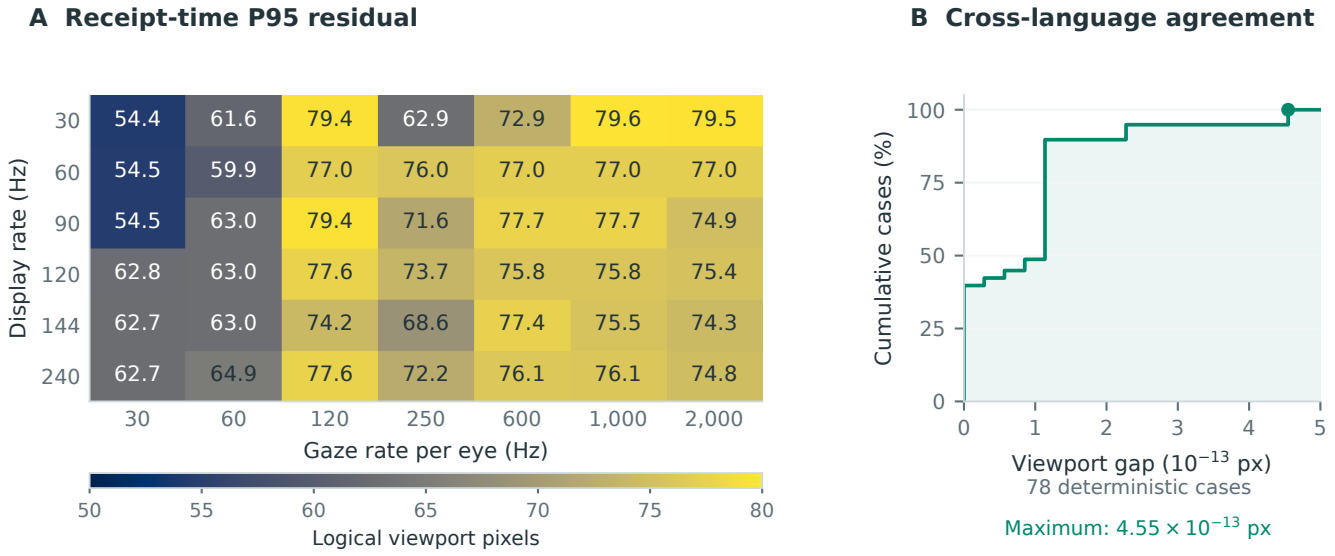


Figure 4: Controlled residuals and implementation agreement. (A) Receipt-time baseline P95 residual for the 42 rate cells at fixed 1280 × 720 viewport and 240 Hz input. (B) Empirical cumulative distribution of the maximum Python–TypeScript viewport discrepancy in each of the 78 cases, including exact-zero cases. The largest gap is 4.55×10^{-13} logical pixels. All cases use deterministic synthetic inputs.

and manuscript source, with installation instructions for the Python and TypeScript artifacts. Machine-readable citation metadata identifies the original GazeSync paper and this technical report.

9 Limitations

The evaluation isolates temporal association and coordinate reconstruction using synthetic trajectories and supplied calibration. Its residuals measure software consistency; device-level accuracy also depends on gaze estimation, exposure and display timing, clock calibration, and camera-to-screen registration. Moving displays require pose measurements aligned with capture time, since pose age alone does not bound motion error.

The scenarios cover scripted interaction and attention patterns. Participant and device studies are needed to assess task-specific attention, adapter behavior across SDKs, and mobile-browser interaction. These studies should report spatial error together with usable coverage and per-eye dropout, using assessment targets separate from calibration. Longer recordings should characterize drift, memory use and latency under the host's persistence and backpressure policy. The present recorder retains bounded in-memory histories, so sustained acquisition requires external storage before rollover.

10 Conclusion

GazeSync reconstructs content-relative gaze during desktop and mobile interaction by joining capture-time observations to recorded presentations. The shared Python and TypeScript contract preserves independent eye channels, coordinate frames and quality decisions through acquisition and replay. Analytic fixtures and synthetic experiments verify frame selection and coordinate reconstruction across acquisition rates and viewport sizes. The GitHub distribution provides researchers with the implementation, adapters and reproducible tests for instrumenting their own applications.

References

[1] Yaxiong Lei, Rishab Talwar, Shijing He, Xinya Gong, Yuheng Wang, Xudong Cai, Zhongliang Guo, and Juan Ye. 2026. GazeSync: A Mobile Eye-Tracking Tool for Analyzing Visual Attention on Dynamically Manipulated Content. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, 1–7. https://doi.org/10.1145/3772363.3798725

[2] Yaxiong Lei, Shijing He, Mohamed Khamis, and Juan Ye. 2023. An End-to-End Review of Gaze Estimation and its Interactive Applications on Handheld Mobile Devices. *ACM Computing Surveys* 56, 2, Article 34, 1–38. https://doi.org/10.1145/3606947

[3] Yaxiong Lei, Yuheng Wang, Tyler Caslin, Alexander Wisowaty, Xu Zhu, Mohamed Khamis, and Juan Ye. 2023. DynamicRead: Exploring Robust Gaze Interaction Methods for Reading on Handheld Mobile Devices under Dynamic Conditions. *Proceedings of the ACM on Human-Computer Interaction* 7, ETRA, Article 158, 1–17. https://doi.org/10.1145/3591127

[4] Yaxiong Lei, Yuheng Wang, Fergus Buchanan, Mingyue Zhao, Yusuke Sugano, Shijing He, Mohamed Khamis, and Juan Ye. 2025. Quantifying the Impact of Motion on 2D Gaze Estimation in Real-World Mobile Interactions. *arXiv preprint* arXiv:2502.10570. https://arxiv.org/abs/2502.10570

[5] Mohammadreza Jamalifard, Yaxiong Lei, Javier Fumanal Idocin, Parastoo Azizinezhad, Tom Foulsham, and Javier Andreu-Perez. 2026. Factor-Informed Uncertainty Distillation for Gaze Estimation. In *Proceedings of the 2026 Symposium on Eye Tracking Research and Applications*, Article 11, 1–7. https://doi.org/10.1145/3797246.3803050

[6] Sophie Stellmach and Raimund Dachselt. 2012. Investigating Gaze-supported Multimodal Pan and Zoom. In *Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA)*. https://imld.de/cnt/uploads/2013/07/2012-ETRA-GazeSuppPanAndZoom.pdf

[7] Christian Kothe, Seyed Yahya Shirazi, Tristan Stenner, David Medine, Chadwick Boulay, Matthew I. Grivich, Fiorenzo Artoni, Tim Mullen, Arnaud Delorme, and Scott Makeig. 2025. The lab streaming layer for synchronized multimodal recording. *Imaging Neuroscience* 3. https://doi.org/10.1162/IMAG.a.136

[8] Tobii Pro. Coordinate systems and Python SDK documentation. https://developer.tobiipro.com/commonconcepts/coordinatesystems.html. Accessed 15 October 2026.

[9] Pupil Labs. Pupil Core developer documentation. https://docs.pupil-labs.com/core/developer/. Accessed 15 October 2026.

[10] SR Research. Python programming with EyeLink. https://www.sr-research.com/support/thread-48-post-47.html. Accessed 15 October 2026.
[11] OpenCV. Camera Calibration and 3D Reconstruction. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html. Accessed 15 October 2026.
[12] Pupil Labs. Pupil Core Terminology: Coordinate System. https://docs.pupil-labs.com/core/terminology/. Accessed 15 October 2026.
[13] W3C. High Resolution Time, Level 3. https://www.w3.org/TR/hr-time-3/. Accessed 15 October 2026.
[14] W3C. Pointer Events, Level 3. https://www.w3.org/TR/pointerevents3/. Accessed 15 October 2026.